

Question

Implement the algorithms using Backtracking approach

Introduction

Backtracking is an algorithm technique used to solve problems incrementally by trying all possible solutions and removing those that fail to satisfy the constraints. It is mainly used for combinatorial and constraint satisfaction problems such as N-Queens, Sudoku, and permutations.

Objective

- To understand the concept of Backtracking
- To implement problems using recursive exploration
- To learn pruning of invalid solutions
- To analyze time complexity

Theory:

Backtracking

Backtracking builds solutions step-by-step and abandons a solution as soon as it determines that it cannot lead to a valid solution.

Key Idea

- Try all possibilities
- Check if the current solution is valid or not valid
- If not valid $\rightarrow$ backtrack undo last step

Example problems

- 1 N-Queens problem
Place N queens on an $N \times N$ board such that no two queens attack each other.
- 2 Permutations of a string
Generate all possible arrangements of characters

Algorithms

1 N-Queens problems

- (i) Start placing queen column by column
- (ii) For each row, check if it is safe
- (iii) If safe, place queen
- (iv) Recur for next column
- (v) If no solution, backtrack and remove queen

(2) permutations

- (i) Fix one character at a time
- (ii) swap with remaining characters
- (iii) Recur for next position
- (iv) Backtrack (swap again)

```

#include <iostream>
#include <vector>
using namespace std;

int issafe(vector<vector<int>> &mat,
           int row, int col)
{
    int n = mat.size();
    int i, j;
    for (i = 0; i < row; i++)
        if (mat[i][col])
            return 0;
    for (i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--)
        if (mat[i][j])
            return 0;
    for (i = row - 1, j = col + 1; j < n && i >= 0; i--, j++)
        if (mat[i][j])
            return 0;
    return 1;
}

void placeQueens (int row, vector<vector<int>> &
                 mat, vector<vector<int>> & result)
{
    int n = mat.size();
    if (row == n) {
        vector<int> ans;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (mat[i][j])
                    { ans.push_back(j + 1); }
            }
        result.push_back(ans);
        return;
    }

    for (int i = 0; i < n; i++)
    {
        if (issafe(mat, row, i))
        {
            mat[row][i] = 1;
            placeQueens (row + 1, mat, result);
            mat[row][i] = 0;
        }
    }
}

```



```

vector<vector<int>> nQueen(int n)
{
    vector<vector<int>> mat(n, vector<int>(n, 0));
    vector<vector<int>> result;
    placeQueens(0, mat, result);
    return result;
}

```

```

int main()
{
    int n=4;
    vector<vector<int>> result=nQueen(n);
    for(auto &ans: result)
    {
        for(auto &i: ans)
        {
            cout<<i<<" ";
        }
        cout<<endl;
    }
    return 0;
}

```

Output:

```

2 4 1 3
3 1 4 2

```

Time complexity = $O(n!)$

conclusion

Backtracking is an effective technique for solving problems where all possible solutions need to be explored. Although it has high time complexity, it ensures correct solutions by systematically exploring and pruning invalid paths.

References

GeeksForGeeks

DSA notes

Introduction to Algorithms

Question

Implement the Approximate Algorithms

Introduction

Approximation algorithms are used to find near-optimal solutions for complex optimization problems where finding the exact solution is computationally expensive (NP-hard problems). These algorithms provide solutions that are close to the optimal with reduced time complexity.

Objective:

- To understand approximation techniques
- To implement approximation algorithms for optimization problem
- To compare approximate and optimal solution
- To analyze performance and efficiency

Theory

Approximation algorithm

An approximation algorithm finds a solution whose value is within a known factor of the optimal solution.

Key concepts

- Approximation Ratio:
Measures how close the solution is to the optimal
- Greedy Approach:
Many approximation algorithms use greedy strategies

Example problems

- vertex cover problem
- Given a graph, find the minimum set of vertices such that every edge is covered
- This problem is NP-hard

Algorithm

vertex cover Approximation

- (1) initialize vertex cover set as empty
- (2) while there are edges in the graph
- (3) pick any edge (u, v)
- (4) Add u and v to the cover-set
- (5) Remove all edges connected to u and v
- (6) Repeat until no edges remain

```

#include <iostream>
#include <vector>
using namespace std;

class Graph {
    int v;
    vector<vector<int>> adj;

public:
    Graph(int v)
    {
        this->v = v;
        adj.resize(v);
    }

    void addEdge(int u, int v)
    {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void vertexCover() {
        vector<bool> visited(v, false);
        for (int u = 0; u < v; u++)
        {
            if (!visited[u])
            {
                for (int v : adj[u])
                {
                    if (!visited[v])
                    {
                        visited[u] = visited[v] = true;
                        break;
                    }
                }
            }
            for (int i = 0; i < v; i++)
            {
                if (visited[i]) cout << i << " ";
            }
        }
    }

    int main() {
        Graph g(7);
        g.addEdge(0, 3);
        g.addEdge(0, 2);
        g.addEdge(2, 3);
        g.addEdge(3, 4);
        g.addEdge(4, 5);
        g.addEdge(5, 6);
        g.addEdge
        g.vertexCover();
        return 0;
    }
}

```

Time complexity
= $O(V + E)$

Conclusion

Approximation algorithms provide efficient solutions for problems where exact computation is not feasible. The vertex cover approximation ensures a solution within twice the optimal size and runs in linear time, making it practical for large inputs.

Reference

GeeksforGeeks

Claude

DSA Notes